`IHttpClientFactory` was introduced in .NET Core 2.1; it makes creating and managing `HttpClient` instances easier and avoids the common pitfalls. In section 33.2 you'll learn how `IHttpClientFactory` achieves this by managing the `HttpClient` handler pipeline. You'll learn how to create named clients to centralize the configuration for calling remote APIs and how to use typed clients to encapsulate the remote service's behavior.

Network glitches are a fact of life when you're working with HTTP APIs, so it's important for you to handle them gracefully. In section 33.3 you'll learn how to use the open-source resilience and fault-tolerance library Polly to handle common transient errors using simple retries, with the possibility for more complex policies.

Finally, in section 33.4 you'll see how you can create your own custom `HttpMessageHandler` handlers managed by `IHttpClientFactory`. You can use custom handlers to implement cross-cutting concerns such as logging, metrics, and authentication, whenever a function needs to execute every time you call an HTTP API. You'll also see how to create a handler that automatically adds an API key to all outgoing requests to an API.

To misquote John Donne, no app is an island, and the most common way of interacting with other apps and services is over HTTP. In .NET, that means using `HttpClient`.

33.1 Calling HTTP APIs: The problem with HttpClient